Karen EL KHOURY

KAREN EL KHOURY

This is the README for the PaintingDesktopAppQt project:

I did step 1 till step 7 and with the optional step 8 with QtDesigner.

I will first give an overview about what I achieved in this project, then give more details and follow the steps of the TP

My code is on My GitHub: https://github.com/KarenKhou/PaintingDesktopAppQt

Overview:

Features

- Drawing Shapes: Line – Rectangle – Ellipse
- Style and Appearance: Pen style: Solid, Dashed, Dotted
- Adjustable line width (2–20 px)
- Confirmation Dialog when closing the app.

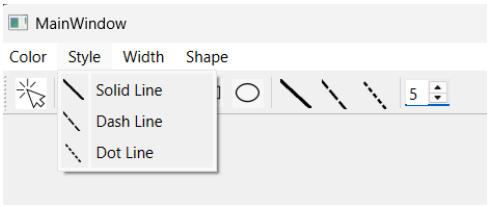
Editing and Selection

- Select shapes with the Select Tool (drawing is disabled when select mode is ON)
- Move shapes by dragging
- Resize using selection handles
- Change color, style, and width of selected shapes

Ui Integration

- Menu bar with menus for Color, Style, Width, and Shape

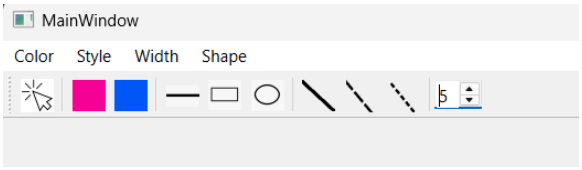
  Menu bar is not ideal for easy use, but it's good for organization when we have many options for each category



- Toolbar with icons for quick access to drawing tools and the "Select" button

  This tool bar is user friendly, practical when we don't have a lot of choices. For bigger apps, its good to have a tool bar for most used or recently used properties.

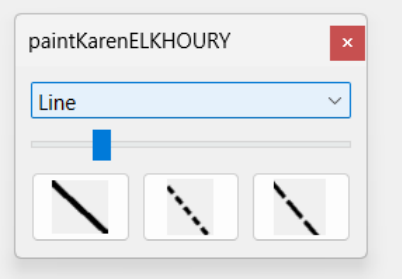
Karen EL KHOURY



- Qt Designer Control panel:

Great on big screens to minimize the mouse movement of the user and keep his/her tools nearby

  - ComboBox for choosing shapes
  - Slider for line width
  - Buttons for style with icons



Implementation details:

Classes

- MainWindow (QMainWindow)
  - Sets up UI with menus, toolbar, and optional designer widgets.
  - Connects actions to canvas slots (used QGroupAction)
  - Manages close confirmation.

- Canvas

  - Defines Slots : selectColor, selectStyle, selectWidth, selectShape, setSelect

  The class has many attributes :

  public attributes like : pinkAction solidLine rectAction in order to use QActionGroup

  As for private attributes:

  - Shape * currentShape : Pointer to the shape currently being drawn by the user.
  - QVector<Shape*> displayList: List of all shapes that have been drawn and are displayed on the canvas, redrawn on every update(). This List contains Rectangles, lines… we apply polymorphism on it.
  - ShapeType currentShapeType: type of shape being drawn (Line, Rectangle, Ellipse)
  - bool selectOption "selection mode" (selecting/editing shapes, user cannot draw when true).
  - Shape * selectedShape: Pointer to the currently selected shape
  - bool objectSelected: Flag that a shape has been selected
  - QPoint lastPos{}: Stores the last mouse position (used for moving shapes).
  - bool resizing: flag for resizing (the user clicked on a resize handle)
  - int handleIndex = -1: Index of the active resize handle (if resizing), or -1 if none
  - Handles mouse events (detailed later in the README)

  - bool moving: flag
  - bool isDrawing : flag
  - QColor currentColor;
  - Qt::PenStyle currentStyle;
  - int currentWidth;

- Shape (Abstract Base Class)
  - Parent Class for All shapes, great for polymorphism
  - Contains definition of enum class ShapeType useful for the instanciation of shapes when drawing ex: switch(currentShapeType){ case ShapeType::Line …
  - Attributes : Each shape has a QPen, a start point and an endpoint and a list of selection handles.
  - Implements setColor, setStyle, width, move, and handleSelected common logic for all shapes
  - Virtual functions for draw(), contains(), resize(), and drawing selection handle useful for polymorphism.

- Derived Shapes: Each shape implements its own way for each virtual method in shapes. This makes it possible to use polymorphism on the displayList <Shape*> heterogene list.
  - Line
  - Rectangle
  - Ellipse

Following the steps in the TP:

Step 1: Create a new project

Step 2: Drawing a line interactively

Implemented mouse event handling: please refer to part "Mouse Events Logic Details" in step 6

Step 3: Specifying graphical attributes

- Color (QActions for Pink and Blue).
- Line style (Solid, Dashed, Dotted).
- Width (QSpinBox).
- + Step 8 optional: added a control panel using QtDesigner

Each category is managed by a single slot (selectColor, selectStyle, selectWidth, selectShape, setSelect).

Inside these slots, we check which action triggered the signal (using sender() or the passed QAction*) to apply the correct attribute.

QActionGroup is used to group related actions, so only one connection per category is required.
Example:

```
auto *colorGroup = new QActionGroup(this);
```

```
colorGroup->addAction(canvas->pinkAction);
colorGroup->addAction(canvas->blueAction);
connect(colorGroup, &QActionGroup::triggered,
      canvas, &Canvas::selectColor);
```

This design avoids code duplication by grouping properties into one slot per category instead of one slot per option.

Step 4: Drawing other shapes

- Introduced ShapeType enum (Line, Rectangle, Ellipse) useful for instantiation of shapes when drawing:

```
switch(currentShapeType){
        case ShapeType::Line:
           currentShape = new Line(
              pen,
              mouseEvent->pos(),
              mouseEvent->pos()
              );
            break;
        case ShapeType::Rectangle:
           currentShape = new Rectangle(
              pen,
              mouseEvent->pos(),
              mouseEvent->pos()
              );
            break;
        case ShapeType::Ellipse:
           currentShape = new Ellipse(
              pen,
              mouseEvent->pos(),
              mouseEvent->pos()
              );
            break;
    }
```

- Depending on the currentShapeType (enum), Canvas creates the appropriate shape subclass.

  We need to do this checking of type only when creating a new shape. Then using polymorphism, each shape will know how to behave accordingly to its overridden methods. no need to check every time for the type.

- Implemented polymorphism with abstract base class Shape and derived classes: Line/Rectangle/Ellipse

OOP and Polymorphism is a good choice for extensibility: in case we want to add another shape we just need to derive from shape and implement its methods.

Step 5: Drawing multiple shapes

- Added a QVector<Shape*> displayList to store all created shapes.
- In paintEvent(), I redraw all shapes from this list.
- Each shape keeps its own QPen (color, width, style) to preserve attributes.

Used polymorphic list of Shape* (Containing heterogeneous objects).

Step 6: Editing the shapes

- Implemented selection mode (selectOption flag).
- Clicking on a shape selects it if contains(point) returns true.

  Hit Test Logic :

  - Line: Check if the click is close to the line segment. Implemented by intersecting the line with a small segment around the mouse point.
  - Rectangle: Check if the click is inside the rectangle's bounding box.Uses QRect::contains()
  - Ellipse : Check if the click is inside the ellipse using the ellipse equation.

- Selection Handles -> Selection handles are small rectangles. we use the same logic to check if the user wants to resize.

- Implemented move() and resize() with selection handles (black squares).
- When a shape is selected:
  - Its attributes (color, width, style) can be modified.
  - It can be dragged or resized by handles.

Mouse Events Logic Details:

mousePressEvent:

Save last position (used to calculate delta in move shape fct).

If selection mode is active :

Loop shapes in displayList.

If mouse inside a shape:

Store the shape as selectedShape

If the click is on a resize handle: start resizing and store handle index

else start moving the shape

Mark the object as selected and return.

Else (drawing mode):

Create a new shape (Line, Rectangle, Ellipse) at the mouse position using current pen (color, style, width).

Set isDrawing = true (flag used in mouse move event)

mouseMoveEvent

As the mouse moves:

If drawing update the endpoint of currentShape
If a shape is selected:

If resizing change the shape size using the handle.

If moving calculate movement (delta = mousePos - lastPos), move the shape, and update lastPos.

mouseReleaseEvent

When the mouse button is released:

Stop moving or resizing (moving = false, resizing = false).

Reset handleIndex =-1

If a shape was being drawn (isDrawing = true):

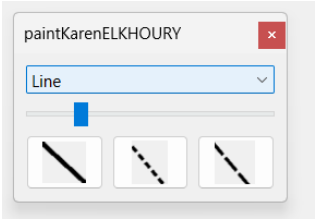
save its endpoint.

Set isDrawing = false.

Add it to displayList

Step 7: Confirmation before exiting

- Override MainWindow::closeEvent(QCloseEvent *event).
- Added QMessageBox asking the user "Do you want to continue?" before quitting.
- If user clicks “No” the event is disregarded, if "Yes", the close event is accepted, this prevents entering in a close event loop



Step 8: Using Qt Designer (optional)

- Integrated additional UI controls:
  - QSlider for line width.
  - QComboBox for shape selection.
  - Buttons for line style with icons.
- Connected Designer widgets to Canvas slots using lambda connections.